



ALLAMA IQBAL OPEN UNIVERSITY
MAIN CAMPUS H-8, ISLAMABAD

PROJECT REPORT

Project Title

Real Time Device to Device Wireless Communication System Using ESP-NOW Protocol

Dual Mode Severity Classification: DBMS Decision Matrix and On Device TinyML Inference

Phase 4

Name: Abdul Rehman

Registration Number: 0000674917

Program: BSCS

Supervisor: Israr Akhter (Slovak Technical University Bratislava)

Table of Contents

1. Background	3
2. Introduction to Phase 4	3
3. Objectives	4
4. Scope	4
5. System Design and Decision Logic.....	5
5.1 Dual Mode Architecture: DBMS and TinyML.....	5
5.2 On Device Decision Tree Classifier	6
5.3 Sender Side Sampling Redesign	6
5.4 Protocols and Design Alternatives	7
6. Implementation.....	8
6.1 Sender Side Firmware Changes.....	8
6.2 Receiver Side Firmware Changes	9
6.3 Offline Model Training and Synthetic Data Generation.....	10
6.4 Server Side TinyML Logging	10
6.5 Database Schema Update.....	11
7. Dashboard Integration.....	11
8. Testing and Results.....	16
9. Limitations	19
10. Future Work	19
12. References.....	20

1. Background

This project is a real time obstacle detection system built around two NodeMCU (ESP8266) boards. One board, called the Sender, sits on a small car and carries an ultrasonic sensor and a PIR motion sensor. When both sensors agree that something is nearby, the Sender reports the detection wirelessly to the second board, called the Receiver, using the ESP-NOW protocol. The Receiver is connected to a home WiFi network and forwards each confirmed detection to a MySQL database through a PHP script running on a local server, and a web dashboard displays this data in an organized and readable way.

Phase 3 added a rule based intelligence layer on top of the original two hop wireless design. Every confirmed detection was placed into a distance based severity zone, normal, warning, or critical, and was separately checked for a timing pattern anomaly by comparing the gap since the previous confirmed detection against the last five recorded gaps. Both signals were then combined by a deterministic decision matrix on the server into one final status, and this classification was made visible on the dashboard with filtering and export support. Phase 3 was explicit that this pattern check was ordinary rule based logic and not a trained model, and it identified genuine machine learning, an Isolation Forest trained separately with scikit-learn, only in the statistical anomaly checks on sensor readings and IP traffic.

Phase 4 builds on this foundation by introducing a second, independent way of deciding severity: a small Decision Tree Classifier trained offline with scikit-learn and converted into plain C++ so it can run directly on the ESP8266 microcontrollers themselves, without needing a server round trip. This phase keeps the Phase 3 server based decision matrix completely intact and adds the on device model beside it, so the project now demonstrates two contrasting approaches to the same classification problem, one running in a MySQL and PHP pipeline and the other running as embedded machine learning inference on constrained hardware.

2. Introduction to Phase 4

In Phase 3, severity was always decided in one place, on the server, after the Receiver forwarded a confirmed detection over WiFi and HTTP. This meant severity could never be known until the network round trip completed, and it meant the buzzer or LED reaction on the Receiver, which did not yet exist in that form, would have depended on the server being reachable.

Phase 4 introduces a physical mode switch on both the Sender and the Receiver, wired independently on each board, so each device decides for itself whether it is operating in DBMS mode or TinyML mode. In DBMS mode, the system behaves exactly as it did in Phase 3, the Receiver posts the confirmed distance, the measured interval, and the pattern anomaly flag to `receive_data.php`, and the server computes and returns the final status. In TinyML mode, the Receiver instead runs its own copy of a Decision Tree Classifier, embedded directly in firmware as `model.h`, on the same three values it just received over ESP-NOW, and reacts immediately through a buzzer and an LED without waiting on any network call. A lightweight, best effort log of that on device prediction is still sent to the server afterward, purely so the dashboard can

show evidence that TinyML mode is running, but this logging step is never on the critical path of the buzzer or LED reaction.

Phase 4 also reworks how the Sender decides when to report a detection. Phase 3 required the object to be held steady for a fixed confirmation duration before anything was sent, which meant a moving object could repeatedly restart its own confirmation and never be reported at all. Phase 4 replaces this with a fixed one second sampling cadence, a quick two reading glitch check, and a duplicate suppression rule based on how far the new distance differs from the last distance actually reported. This makes the system considerably more responsive to a moving object while keeping the same overall wireless message rate.

3. Objectives

- To let the Sender and the Receiver each choose independently between a server based decision matrix and an on device machine learning model for deciding severity, through a physical switch on each board.
- To train a Decision Tree Classifier offline on distance, measured interval, and pattern anomaly, and to convert it into plain C++ so it can run directly on the ESP8266 without needing scikit-learn, Python, or a network connection at inference time.
- To redesign the Sender's reporting logic around a fixed sampling cadence and a duplicate suppression check, so a moving object is reported responsively instead of only after a fixed confirmation duration.
- To make the Receiver react to severity immediately in TinyML mode through a buzzer and an LED, independent of the Sender's own switch state and independent of network availability.
- To keep a clear, separately logged record of every on device prediction, distinct from the server computed detection_events table, so the two classification paths can be inspected and compared honestly rather than mixed together.
- To extend the dashboard so the trained model's offline test performance and its live, on device predictions are both visible, filterable, and exportable.

4. Scope

This phase covers the following parts of the system.

- Firmware changes on the Sender NodeMCU, replacing the fixed confirmation duration with a one second sampling cadence, a glitch rejecting recheck, and a distance based duplicate suppression rule, and adding a mode switch that determines whether the Sender also runs its own copy of the Decision Tree and includes that prediction in the packet it sends.
- Firmware changes on the Receiver NodeMCU, adding its own independent mode switch, a buzzer and a severity indicating LED, and two mode exclusive reaction paths, one that forwards data to the server as in Phase 3, and one that runs the Decision Tree locally and reacts without any network dependency.

- An offline training pipeline in Python, using scikit-learn to train the Decision Tree Classifier on labeled detection data and micromlgen to export the trained tree as C++ source code shared by both boards.
- A synthetic, clearly labeled test data generator, used only to give the dashboard, its filters and exports, and the trained model enough balanced volume to be demonstrated and evaluated properly.
- A new database table, `tinyml_predictions`, and a dedicated logging endpoint, `log_tinyml_result.php`, kept structurally separate from the `detection_events` table and its DBMS pipeline.
- Dashboard updates, adding a Machine Learning page that shows the offline training performance and a live, filterable, exportable log of on device predictions.

This phase does not change the Phase 3 decision matrix itself, and it does not attempt to make the Decision Tree learn anything beyond the same three features already used by the rule based check. The honesty distinction that Phase 3 drew between deterministic logic and trained machine learning is carried forward here between the server side decision matrix, which remains rule based, and the Decision Tree, which is a genuine, if intentionally small, supervised model.

5. System Design and Decision Logic

5.1 Dual Mode Architecture: DBMS and TinyML

Each board now reads its own mode switch independently, using the microcontroller's internal pull up resistor so no external resistor is required. The Sender reads its switch on pin D4, and the Receiver reads its switch on pin D7. An open switch reads HIGH and selects DBMS mode, and a switch closed to ground reads LOW and selects TinyML mode. The two switches are not linked to each other in any way, so it is entirely possible, and useful for testing, to have the Sender computing its own TinyML prediction purely for logging purposes while the Receiver is still in DBMS mode and relying on the server, or the reverse.

On the Receiver, the two modes are kept mode exclusive on the output hardware. DBMS mode uses only the Green and Red LEDs, exactly as in Phase 3, where Green means idle or no current detection and Red means data is currently being received, and this pair carries no severity information of its own. TinyML mode uses only the Yellow LED and the buzzer, and the Green and Red LEDs are forced off for the entire time TinyML mode is active. This separation is enforced on every loop iteration, not only when new data arrives, so flipping the switch takes effect immediately even with no detections in progress.

The severity reaction itself in TinyML mode is simple and immediate. A normal prediction blinks the Yellow LED on for two seconds before turning it off, with the buzzer silent throughout. A warning prediction beeps the buzzer once. A critical prediction beeps the buzzer twice. Because this reaction happens directly from the Receiver's own on device prediction, it never depends on the Sender's own switch state or on whatever prediction, if any, the Sender computed for the same event.

5.2 On Device Decision Tree Classifier

The classifier is a scikit-learn `DecisionTreeClassifier`, trained offline on three features already produced by the existing pipeline for every confirmed detection, the ultrasonic distance in centimetres, the measured interval in seconds since the previous confirmed detection, and the pattern anomaly flag inherited unchanged from Phase 3. Its target label is `final_status`, the same three class outcome, normal, warning, or critical, already used throughout the project. The tree depth is deliberately capped at five so the exported C++ stays small and fast enough for an ESP8266.

Once trained, the tree is converted to plain C++ with the `micromlgen` library, which ports several scikit-learn model types, Decision Trees among them, into a small, dependency free class under the `Eloquent::ML::Port` namespace. The generated file, `model.h`, exposes a `predict` method that takes a three element feature array and returns an integer class index, and a `predictLabel` or `idxToLabel` helper that turns that index into a readable string. This same header file is included unchanged in both `sender.ino` and `receiver.ino`, so both boards run an identical copy of the trained model.

An important and worth stating honestly result of this training run is that the exported tree ended up making its decision purely from the distance feature, splitting only at 7.5 centimetres and 14.5 centimetres, the same boundaries as the Phase 3 severity zones, and never branching on the measured interval or the pattern anomaly flag at all. This is a direct consequence of how the training labels were generated rather than a limitation of the Decision Tree algorithm itself. The synthetic data generator used to build the offline training set assigns `final_status` purely from the distance zone, with no exception for a pattern anomaly, specifically so that the three input features would not send the model conflicting signals during training. Since the interval and the anomaly flag carry no information about the label under that generation rule, an accuracy maximizing tree has every reason to ignore them and none to split on them. The two extra features remain part of the `predict` function's signature so that live sensor timing data can be fed into the model exactly as designed, and so a future retraining pass on a differently labeled dataset, one where the anomaly flag is allowed to influence the target as it does in the live server side decision matrix, could still make use of them without any firmware change.

5.3 Sender Side Sampling Redesign

Phase 3's Sender required the ultrasonic and motion sensors to agree and the distance to remain within tolerance for a full second before anything was reported, and any interruption during that window restarted the wait. This worked adequately for a mostly stationary target but meant a continuously moving object could, in the worst case, never hold still long enough to be confirmed at all.

Phase 4 replaces this with a fixed sampling cadence of one sample per second. At each sample, if both sensors currently agree that something is nearby, the Sender takes one additional reading one hundred milliseconds later purely to reject a single noisy glitch, and only proceeds if the two readings agree within a three centimetre tolerance. The resulting distance is then compared against the distance that was actually reported last time. If the two are within the same three centimetre tolerance, the sample is treated as an unchanged duplicate and nothing is sent. If the

distance has genuinely moved beyond that tolerance, it is reported immediately, without waiting for a fixed confirmation window. This makes the Sender far more responsive to a moving object while still suppressing the flood of near identical messages a perfectly still object would otherwise generate every second.

Because the sampling cadence changed from Phase 3's variable, sometimes multi second, confirmation delay to a steady one second rhythm, the expected interval used by the pattern anomaly check was updated from roughly four seconds to one second, while the same twenty five percent tolerance, five event rolling history, and three out of five deviation threshold from Phase 3 were kept unchanged in concept.

5.4 Protocols and Design Alternatives

The communication protocols established in Phase 3, ESP-NOW between the two boards at the data link layer and WiFi with TCP, IP, and HTTP between the Receiver and the local server at the higher layers, are unchanged in Phase 4 and are not repeated here in full. The table below focuses on the two mechanisms that are new to this phase, on device inference and its offline training and export pipeline.

Table 1: Functions performed by the system, selected mechanisms, alternative approaches considered, and the reasons for rejecting those alternatives.

Function Performed	Mechanism Used	Alternative(s) Considered	Reason Alternative Not Chosen
On device severity inference	Decision Tree Classifier (scikit-learn), exported to C++ with micromlgen and compiled directly into sender.ino and receiver.ino	A small neural network exported with TensorFlow Lite for Microcontrollers	A neural network needs a runtime interpreter and more flash and RAM than a shallow Decision Tree, and offers no accuracy benefit on a three feature, three class problem this simple
Training the offline model	scikit-learn DecisionTreeClassifier, capped at a maximum depth of five, trained on a labeled MySQL dataset	Training directly from a rule expressed in firmware, with no learned model at all	Would not demonstrate genuine supervised learning or produce a model with measurable test accuracy, precision, recall, and a confusion matrix the way a trained classifier does

Getting a trained model onto the microcontroller	micromlgen, which ports a fitted scikit-learn model into a dependency free C++ class	Running scikit-learn or a Python interpreter on the ESP8266 itself	The ESP8266 has neither the flash space nor the processing power to host a Python runtime or the scikit-learn library, so inference must be exported to native C++ ahead of time
Logging what the on device model predicted	A dedicated tinyml_predictions table, written to by log_tinyml_result.php after the buzzer and LED have already reacted	Writing on device predictions into the same detection_events table used by the DBMS decision matrix	Mixing rule based and learned classification results into one table would make it impossible to tell afterward which severity value came from which mechanism, undermining the honesty distinction the project has kept since Phase 3
Deciding whether a board uses DBMS or TinyML logic	A physical switch per board, read with the microcontroller's internal pull up resistor	A single shared switch or a software configuration flag set once at compile time	A shared switch would prevent testing the Sender and Receiver in different modes at the same time, and a compile time flag would need a full re-upload of firmware just to change modes during testing

6. Implementation

6.1 Sender Side Firmware Changes

The Sender firmware now defines a `SAMPLE_INTERVAL_MS` constant of one thousand milliseconds and, once every interval, checks whether both the ultrasonic and motion sensors currently agree. If they do, it performs the hundred millisecond glitch recheck described earlier,

and compares the confirmed distance against `lastReportedDistance` using a three centimetre `DISTANCE_TOLERANCE_CM` constant. A genuinely new distance is reported over ESP-NOW; a duplicate is logged to the serial monitor and silently skipped. The pattern anomaly evaluation function, `evaluateAndRecordInterval`, is unchanged in its internal logic from Phase 3, only its expected interval constant was updated from four seconds to one second to match the new cadence.

Before sending, the Sender reads its own mode switch on pin D4. If the switch indicates TinyML mode, the Sender builds the same three element feature array, distance, measured interval, and pattern anomaly, that the server side decision matrix would also see, runs it through the shared `tinyMLModel.predict` method, and includes the resulting class index in the outgoing ESP-NOW packet alongside a `tinyMLMode` flag. If the switch indicates DBMS mode, this on device prediction step is skipped entirely and the outgoing packet's `tinyMLSeverity` field is left at zero, unused. In either mode the same distance, pattern anomaly flag, and measured interval are always included in the packet, since the Receiver needs them regardless of which mode it itself is in.

It is worth noting explicitly that the Sender never calls the server directly in either mode. It only ever transmits over ESP-NOW to the Receiver; all HTTP communication with the PHP backend, in both DBMS mode and TinyML mode, is the Receiver's responsibility.

6.2 Receiver Side Firmware Changes

The Receiver firmware adds its own mode switch on pin D7, read independently of whatever `tinyMLMode` flag arrived from the Sender in the incoming packet. Work is handed off from the ESP-NOW receive callback to the main loop through a small set of pending variables, since callbacks are kept short, and the main loop checks its own mode switch fresh on every pass, so toggling the switch takes effect immediately even between detections.

In DBMS mode, the Receiver's behaviour is unchanged from Phase 3, it posts the confirmed distance, the measured interval, and the pattern anomaly flag to `receive_data.php` as a JSON body, reads back the `final_status` field from the server's JSON response through a small purpose built string extractor, and prints that server computed severity to the serial monitor. The Yellow LED and buzzer are forced off for the entire time DBMS mode is active, and only the Green or Red LED reflects whether data is currently being received.

In TinyML mode, the Receiver skips the HTTP call to the server entirely for the purpose of deciding severity. It builds the same three element feature array from what it just received over ESP-NOW and evaluates its own local copy of the Decision Tree, obtaining a predicted class and its readable label independent of whatever prediction the Sender itself may have computed for the same event. The buzzer and Yellow LED react immediately to this local result through the `reactToSeverity` function. Only after this immediate reaction does the Receiver attempt a best effort log of the prediction to `log_tinyml_result.php`; if there is no WiFi connectivity at that moment, this call fails silently and is skipped, since it was never required for the physical buzzer and LED response to happen.

6.3 Offline Model Training and Synthetic Data Generation

Because a newly deployed system has too few real detection events to train a meaningful classifier, `generate_test_data.py` produces a large, clearly synthetic dataset purely for this purpose, and every comment in the script is explicit that none of it is real sensor data. Earlier versions of this generator picked timestamps uniformly at random across the whole project date range, which produced measured intervals far longer than the sampling cadence, made almost every synthetic event look like a pattern anomaly, and starved the normal class of clean, non-anomalous examples. The current version instead builds a series of separate simulated driving sessions, each starting at its own random moment within the overall date range and then producing a run of closely spaced events with realistic jitter and a small deliberately injected anomaly rate, which is a far more faithful approximation of how a real Sender behaves while continuously powered on.

A deliberate design decision in this generator is that `final_status` is computed purely from the distance zone, with no exception made for a pattern anomaly, unlike the server side decision matrix from Phase 3, which does allow a pattern anomaly to escalate a warning distance reading to critical or a normal distance reading to warning. This choice was made specifically for the TinyML training set, so that the pattern anomaly and interval features would never contradict the distance based label during training, letting the Decision Tree learn a clean, purely distance driven rule that generalizes exactly as intended for an embedded classifier with only three simple features.

`train_tinyml_severity.py` connects to the same MySQL database, pulls the ultrasonic distance, measured interval, and pattern anomaly flag for every confirmed event alongside its `final_status` label, and performs a stratified eighty to twenty train and test split. It fits a `DecisionTreeClassifier` with a maximum depth of five, evaluates it on the held out test set with accuracy, a confusion matrix, and a full precision, recall, and F1 classification report, and writes all of these numbers to `ml_metrics.json` so the dashboard can display genuine training performance rather than placeholder figures. It then refits an identically configured tree on integer encoded labels purely so `micromlgen` can export readable class names, and writes the resulting C++ class to `model.h`, ready to be copied into both `sender.ino` and `receiver.ino`.

6.4 Server Side TinyML Logging

`log_tinyml_result.php` is intentionally the simplest endpoint in the entire backend. It performs no classification of its own, unlike `receive_data.php`, and exists only to record a prediction a device has already made and already reacted to. It validates that `device_role` is either sender or receiver, that a distance value is present, and that `predicted_severity` is one of the three known classes, then inserts a single row into `tinyml_predictions` using a prepared statement. Because this call happens strictly after the buzzer and LED have already responded on the Receiver, a failure of this endpoint, for example due to no WiFi connectivity at that moment, has no effect whatsoever on the system's real time behaviour, and only reduces what the dashboard can show after the fact.

6.5 Database Schema Update

A new table, `tinyml_predictions`, was added specifically for this phase, deliberately structured to remain independent of `detection_events` and the columns Phase 3 added to it. It stores which device role logged the prediction, the reading value, the measured interval, the pattern anomaly flag, the predicted severity, and a server side timestamp. Keeping this as its own table, rather than adding TinyML columns onto `detection_events`, means the dashboard can always show unambiguously which rows came from the rule based DBMS pipeline and which came from genuine on device inference, without the two ever being confused with one another.

Table 2: Database schema defining the columns, data types, and purposes of the fields used to store model prediction records.

Column	Type	Purpose
<code>prediction_id</code>	INT, primary key, auto increment	Unique identifier for each logged prediction
<code>device_role</code>	ENUM('sender', 'receiver')	Which board produced this on device prediction
<code>reading_value</code>	FLOAT	The ultrasonic distance in centimetres used as a model input
<code>measured_interval_sec</code>	FLOAT, nullable	The measured gap since the previous confirmed detection, null on a device's very first prediction
<code>is_pattern_anomaly</code>	BOOLEAN	The pattern anomaly flag used as a model input, computed identically to the Phase 3 rule
<code>predicted_severity</code>	ENUM('normal', 'warning', 'critical')	The class the Decision Tree returned for this input
<code>logged_at</code>	TIMESTAMP, default current timestamp	When the server received this log entry, indexed for filtering and ordering

7. Dashboard Integration

The Overview page continues to show the most recently confirmed distance with its severity badge, live counts for the last minute and the last hour, and a running total across the entire database. As of the testing snapshot taken for this report, the system had accumulated thirty thousand and ten confirmed detection events, of which ten had occurred that day, with ten events in the last hour and six of those classified critical.

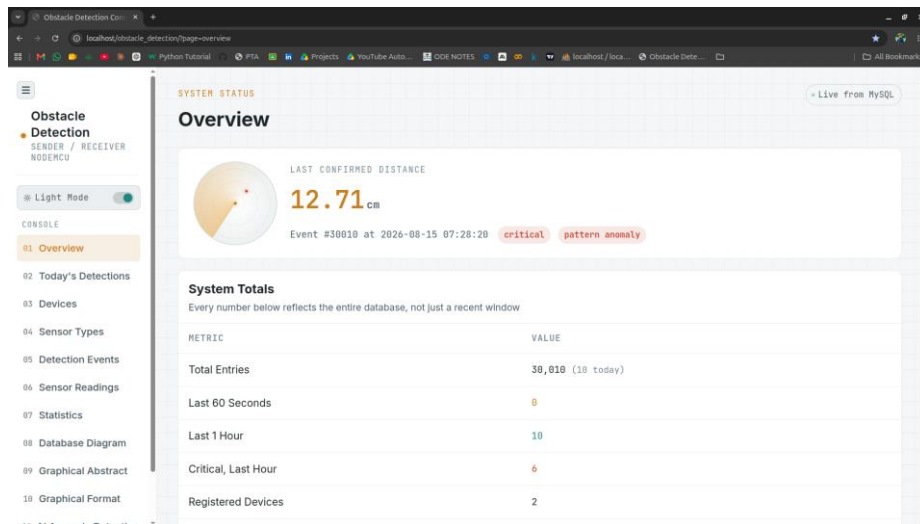


Figure 1: Overview page showing the last confirmed distance, its severity and pattern anomaly badges, and live system totals.

Scrolling further down the same page, the system totals table breaks the dataset down by device, sensor type, and severity class. All thirty thousand and ten entries in this snapshot were attributed to the Sender car, since the bulk synthetic dataset generated for model training was inserted under the Sender's device record, matching how a real deployment would also log almost all of its traffic from the Sender. The severity distribution shows twenty one thousand and sixty five normal readings, six thousand six hundred and sixty seven warning readings, and two thousand two hundred and seventy eight critical readings, with sixty three timing pattern anomalies recorded across the full first of April to fifteenth of August twenty twenty six date range.

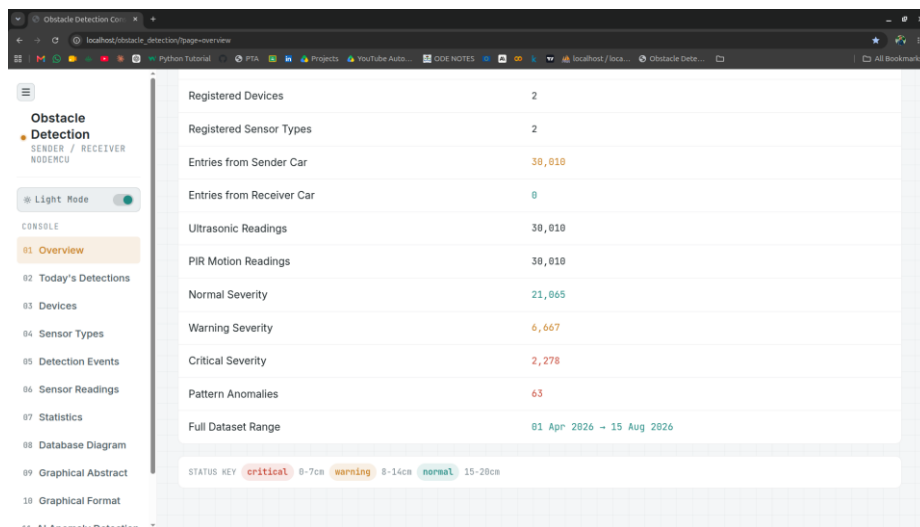


Figure 2: System totals showing the full severity and pattern anomaly breakdown across the entire database.

The Today's Detections page lists every confirmed event from the current day as a live log, most recent first, and reset automatically at midnight. On the day this snapshot was taken, ten events had already been confirmed, with a mixture of critical, warning, and normal classifications consistent with the manual DBMS mode testing carried out for this phase.

Obstacle Detection
SENDER / RECEIVER
NODEMCU

Light Mode

CONSOLE

- 01 Overview
- 02 Today's Detections
- 03 Devices
- 04 Sensor Types
- 05 Detection Events
- 06 Sensor Readings
- 07 Statistics
- 08 Database Diagram
- 09 Graphical Abstract
- 10 Graphical Format
- 11 AI Anomaly Detection

LIVE LOG

Today's Detections

Saturday, 15 August 2026
Resets automatically at midnight — 10 entries so far today [Full History](#)

EVENT ID	DEVICE	DISTANCE	STATUS	TIME
#30010	Sender Car	12.71 cm	critical	2026-08-15 07:28:20
#30009	Sender Car	6.53 cm	critical	2026-08-15 07:28:15
#30008	Sender Car	15.50 cm	normal	2026-08-15 07:27:34
#30007	Sender Car	11.82 cm	warning	2026-08-15 07:27:30
#30006	Sender Car	15.18 cm	normal	2026-08-15 07:27:22
#30005	Sender Car	9.45 cm	critical	2026-08-15 07:26:12
#30004	Sender Car	6.52 cm	critical	2026-08-15 07:24:45
#30003	Sender Car	12.11 cm	critical	2026-08-15 07:24:39
#30002	Sender Car	15.18 cm	warning	2026-08-15 07:24:38

Figure 3: Today's Detections page, showing the live log of confirmed events for the current day.

The Sensor Readings page, unchanged in structure from Phase 3, continues to show one row per individual sensor value, joined against its parent detection event. With thirty thousand and ten confirmed events, each contributing one ultrasonic reading and one PIR motion reading, this page reports sixty thousand and twenty matching rows with no filter applied, confirming the expected two readings per event relationship.

Obstacle Detection
SENDER / RECEIVER
NODEMCU

Light Mode

CONSOLE

- 01 Overview
- 02 Today's Detections
- 03 Devices
- 04 Sensor Types
- 05 Detection Events
- 06 Sensor Readings
- 07 Statistics
- 08 Database Diagram
- 09 Graphical Abstract
- 10 Graphical Format
- 11 AI Anomaly Detection

TABLE 04, JOINED VIEW

Sensor Readings

Search This Table
Filter by device, sensor, date, time, or distance to find a specific pattern

All devices All sensors Date mm/dd/yyyy Time --:--:-- Distance bucket (cm)

None or custom range (cm) mi ms [Apply Filter](#)

Export This Result [CSV](#) [Text](#) [HTML](#) [PDF](#)

MATCHING READINGS
60,020 across the whole database (no filter applied)

READING ID	EVENT ID	DEVICE	SENSOR	VALUE	TIME
#60020	#30010	Sender Car	PIR Motion	1.00 boolean	2026-08-15 07:28:20
#60019	#30010	Sender Car	Ultrasonic	12.71 cm	2026-08-15 07:28:20
#60018	#30009	Sender Car	PIR Motion	1.00 boolean	2026-08-15 07:28:15
#60017	#30009	Sender Car	Ultrasonic	6.53 cm	2026-08-15 07:28:15

Figure 4: Sensor Readings page showing the joined ultrasonic and motion readings for each confirmed event.

The AI Anomaly Detection page keeps the same three part structure introduced in Phase 3, a rule based pattern check table drawn from detection_events, followed by the two Isolation Forest based statistical checks. The rule based table is a useful place to observe how the server side decision matrix, unchanged from Phase 3, still allows a pattern anomaly to escalate severity, for example event thirty thousand and ten was in the warning distance zone but carried a final status of critical once its pattern anomaly flag was factored in, which is a direct contrast with the purely distance driven label used to train the offline Decision Tree in Section 6.3.

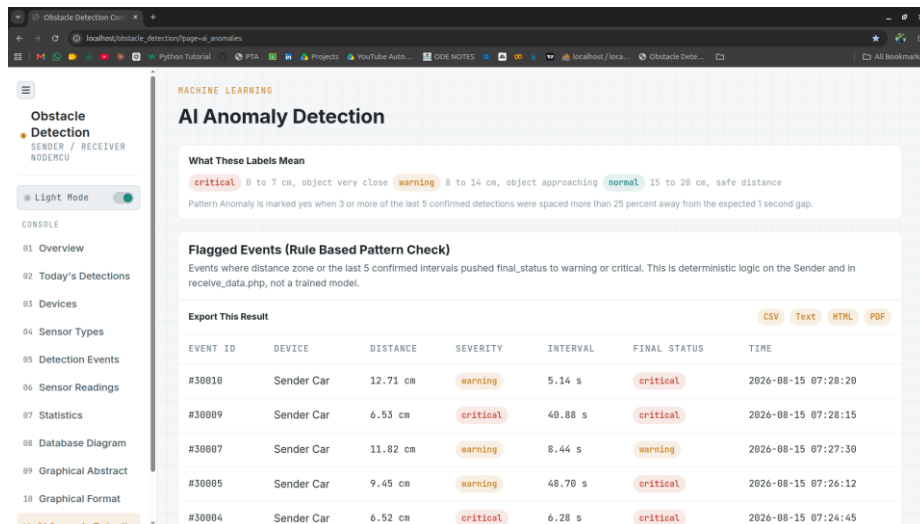


Figure 5: AI Anomaly Detection page, rule based pattern check table, showing the server side decision matrix still escalating severity when a pattern anomaly accompanies a warning zone distance.

The two Isolation Forest sections beneath it, Flagged Sensor Readings and Flagged IP Addresses, remained empty in this snapshot, each showing its documented placeholder message pointing to the separate training scripts that must be run before either check produces results. This is consistent with Phase 3's design, where these two checks are intentionally decoupled from the live detection pipeline and only ever reflect results already written to the database by their own offline training runs.

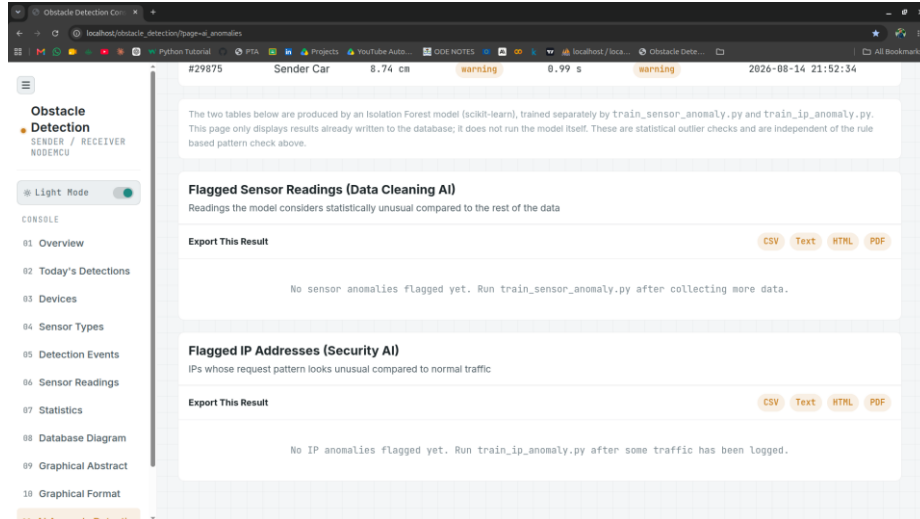


Figure 6: AI Anomaly Detection page, Isolation Forest based sensor reading and IP address checks, both awaiting their own offline training run.

The new Machine Learning page is specific to Phase 4. Its upper section explains plainly what the page shows, that the Decision Tree runs directly on the microcontroller with no server involved at prediction time, and that the Receiver's buzzer and Yellow LED only react when both boards' switches are set to TinyML at the same time. Beneath this explanation, the training performance panel reads its figures directly from ml_metrics.json, reporting a test set accuracy of one hundred percent on thirty thousand total rows split into twenty four thousand training

rows and six thousand test rows, with a maximum tree depth of five, trained on the fifteenth of August twenty twenty six.

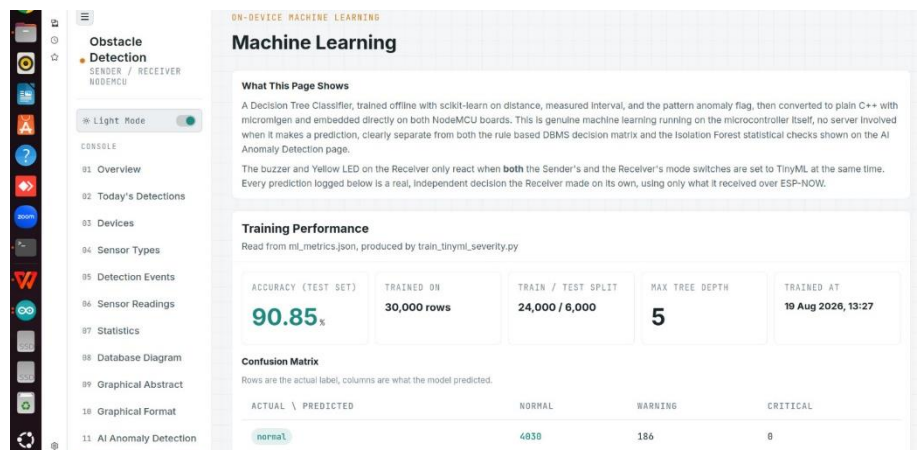


Figure 7: Machine Learning page, explanation panel and training performance summary read from ml_metrics.json.

Scrolling further down the same page, the confusion matrix and classification report make the reason for that one hundred percent figure visible rather than simply asserting it. Every one of the four thousand two hundred and thirteen normal, one thousand three hundred and thirty three warning, and four hundred and fifty four critical test rows was placed on the diagonal of the confusion matrix, giving a precision, recall, and F1 score of exactly one point zero zero zero for all three classes. As discussed in Section 5.2, this perfect separation reflects the fact that the synthetic training labels are a deterministic function of distance alone, and a shallow Decision Tree finds the two correct thresholds with no difficulty; it should be read as evidence that the model faithfully reproduced the rule it was trained on, not as evidence that the model would necessarily perform this well on genuinely noisy, mislabeled, or borderline real world sensor data.

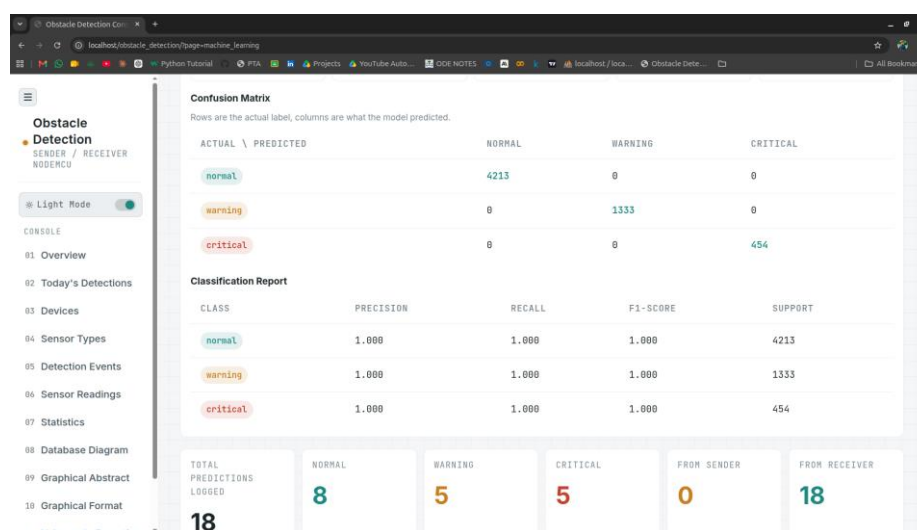
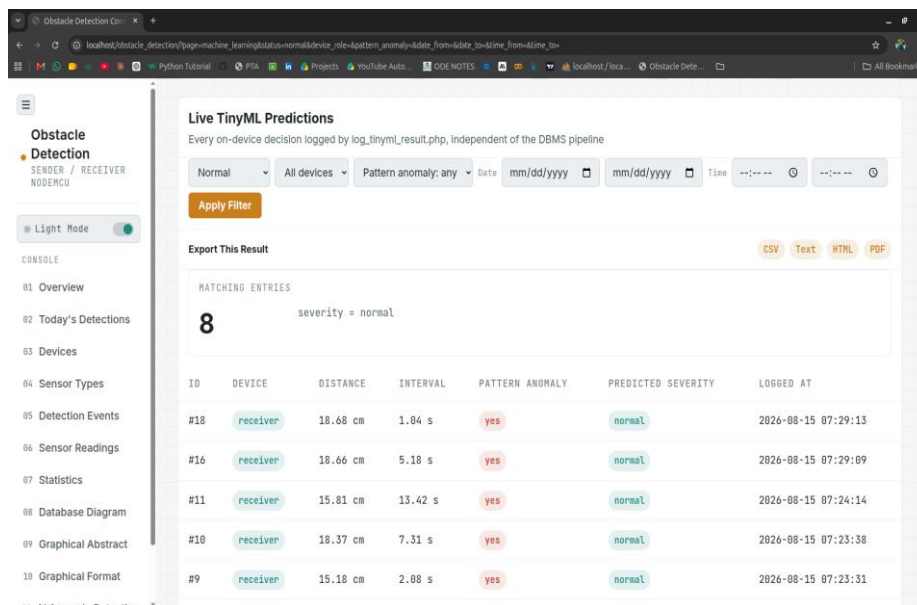


Figure 8: Machine Learning page, confusion matrix and per class classification report, showing perfect separation on the synthetic test set.

Beneath the offline training results, the Live TinyML Predictions panel lists every row actually written to `tinyml_predictions` by `log_tinyml_result.php`, independent of the DBMS pipeline, with the same filtering, exporting, and pagination pattern used elsewhere on the dashboard. At the time of this snapshot, eighteen predictions had been logged in total, eight normal, five warning, and five critical, and every single one of them had been logged by the Receiver; none had come from the Sender, which matches the firmware design in Section 6.1, since the Sender computes its own prediction purely for inclusion in the outgoing ESP-NOW packet and never calls `log_tinyml_result.php` itself.

8. Testing and Results

To verify that the on device Decision Tree behaves consistently with the offline evaluation reported in Section 7, the Live TinyML Predictions table was filtered by each severity class in turn. Filtering to Normal returned exactly eight matching entries, all logged by the Receiver, each showing a distance at or above fifteen centimetres, consistent with the boundary the tree actually learned to split on.



Live TinyML Predictions
Every on-device decision logged by `log_tinyml_result.php`, independent of the DBMS pipeline

Normal | All devices | Pattern anomaly: any | Date: mm/dd/yyyy | Time: --:--:--

Apply Filter

Export This Result: CSV, Text, HTML, PDF

MATCHING ENTRIES
8 severity = normal

ID	DEVICE	DISTANCE	INTERVAL	PATTERN ANOMALY	PREDICTED SEVERITY	LOGGED AT
#18	receiver	18.68 cm	1.84 s	yes	normal	2026-08-15 07:29:13
#16	receiver	18.66 cm	5.18 s	yes	normal	2026-08-15 07:29:09
#11	receiver	15.81 cm	13.42 s	yes	normal	2026-08-15 07:24:14
#10	receiver	18.37 cm	7.31 s	yes	normal	2026-08-15 07:23:38
#9	receiver	15.18 cm	2.88 s	yes	normal	2026-08-15 07:23:31

Figure 9: Live TinyML Predictions filtered to Normal severity, returning eight matching entries.

Filtering to Warning returned exactly five matching entries, each with a distance between roughly eight and fourteen centimetres, and filtering to Critical returned exactly five matching entries, each with a distance below roughly eight centimetres. The three severity filters together account for all eighteen logged predictions, eight plus five plus five, confirming that the filter, the count shown in the Matching Entries panel, and the underlying table agree with one another.

Obstacle Detection
SENDER / RECEIVER
NODENCU

Light Mode

CONSOLE

- 01 Overview
- 02 Today's Detections
- 03 Devices
- 04 Sensor Types
- 05 Detection Events
- 06 Sensor Readings
- 07 Statistics
- 08 Database Diagram
- 09 Graphical Abstract
- 10 Graphical Format
- 11 All Anomaly Detections

Live TinyML Predictions
Every on-device decision logged by log_tinyml_result.php, independent of the DBMS pipeline

Warning All devices Pattern anomaly: any Date mm/dd/yyyy Time --:--:--

Apply Filter

Export This Result CSV Text HTML PDF

MATCHING ENTRIES
5 severity = warning

ID	DEVICE	DISTANCE	INTERVAL	PATTERN ANOMALY	PREDICTED SEVERITY	LOGGED AT
#17	receiver	14.20 cm	3.11 s	yes	warning	2026-08-15 07:29:10
#15	receiver	8.27 cm	2.86 s	yes	warning	2026-08-15 07:29:02
#12	receiver	9.76 cm	4.22 s	yes	warning	2026-08-15 07:28:34
#5	receiver	13.87 cm	13.45 s	yes	warning	2026-08-15 07:22:57
#3	receiver	14.22 cm	5.17 s	yes	warning	2026-08-15 07:21:38

Showing 1-5 of 5 rows

First Prev Page 1 / 1 Next Last

Figure 10: Live TinyML Predictions filtered to Warning severity, returning five matching entries.

Obstacle Detection
SENDER / RECEIVER
NODENCU

Light Mode

CONSOLE

- 01 Overview
- 02 Today's Detections
- 03 Devices
- 04 Sensor Types
- 05 Detection Events
- 06 Sensor Readings
- 07 Statistics
- 08 Database Diagram
- 09 Graphical Abstract
- 10 Graphical Format
- 11 All Anomaly Detections

Live TinyML Predictions
Every on-device decision logged by log_tinyml_result.php, independent of the DBMS pipeline

Critical All devices Pattern anomaly: any Date mm/dd/yyyy Time --:--:--

Apply Filter

Export This Result CSV Text HTML PDF

MATCHING ENTRIES
5 severity = critical

ID	DEVICE	DISTANCE	INTERVAL	PATTERN ANOMALY	PREDICTED SEVERITY	LOGGED AT
#14	receiver	4.18 cm	19.73 s	yes	critical	2026-08-15 07:29:00
#13	receiver	4.18 cm	1.03 s	yes	critical	2026-08-15 07:28:35
#8	receiver	7.39 cm	7.27 s	yes	critical	2026-08-15 07:23:27
#6	receiver	5.63 cm	19.00 s	yes	critical	2026-08-15 07:23:17
#4	receiver	3.89 cm	5.18 s	yes	critical	2026-08-15 07:21:48

Figure 11: Live TinyML Predictions filtered to Critical severity, returning five matching entries.

The pattern anomaly filter was tested in the same way. Filtering to pattern anomaly equal to yes returned seventeen of the eighteen logged predictions, and filtering to pattern anomaly equal to no returned exactly one, together again accounting for the full eighteen rows. This heavy skew toward flagged anomalies is expected for this particular test session, since the predictions were generated by manually triggering detections at irregular, human paced intervals rather than by a continuously moving object sampled every second as the pattern check assumes, and it is discussed further as a limitation in Section 9.

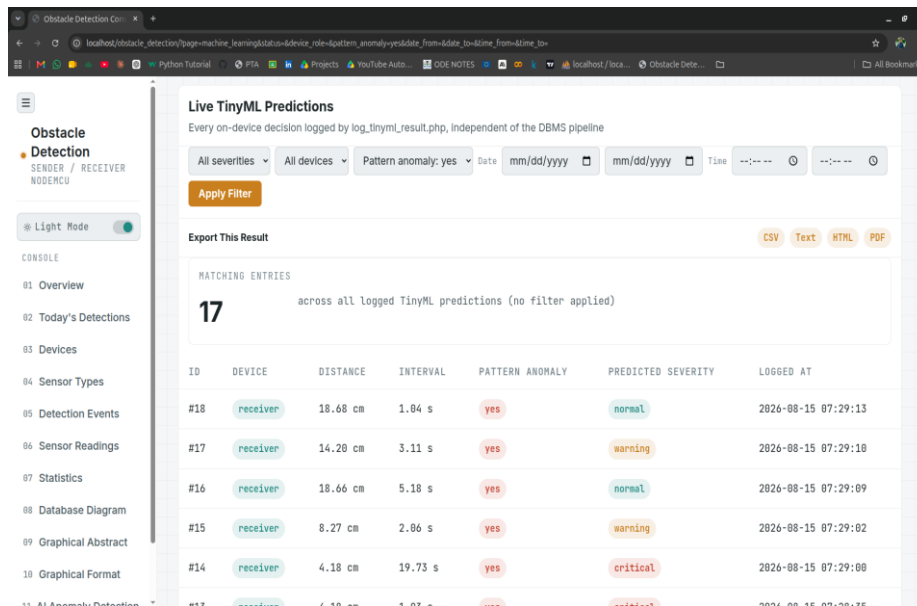


Figure 12: Live TinyML Predictions filtered to pattern anomaly equal to yes, returning seventeen of eighteen entries.

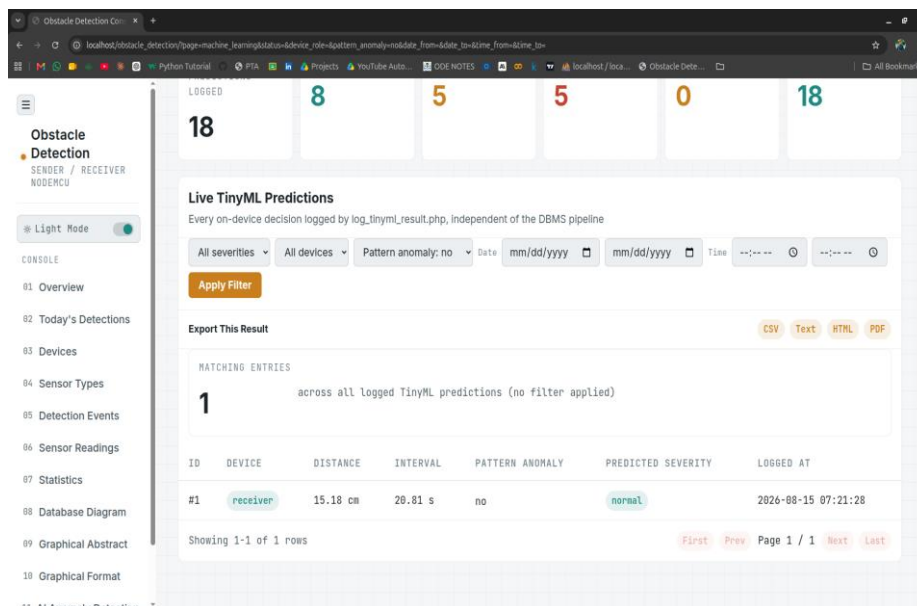


Figure 13: Live TinyML Predictions filtered to pattern anomaly equal to no, returning the single remaining entry.

Across every filtered view, the distance, interval, and predicted severity shown for each individual row remain visibly consistent with the thresholds the offline confusion matrix already confirmed, which supports the conclusion that the C++ export produced by micromlgen is behaving identically to the scikit-learn model it was converted from, and that the Receiver's on device inference path is functioning correctly end to end from ESP-NOW reception through to a logged, filterable dashboard row.

9. Limitations

- The Decision Tree's perfect one hundred percent test accuracy is a direct consequence of the synthetic training labels being a deterministic function of distance alone. It should not be read as a general claim about how the model would perform on noisier, real world sensor data with borderline or mislabeled cases.
- The tree ended up using only the distance feature, effectively rediscovering the same fixed thresholds as the Phase 3 severity zones on its own. The measured interval and pattern anomaly features are present in the model's input signature but currently carry no influence over its predictions, because the training labels were deliberately generated without letting those two features affect the class.
- The synthetic training dataset's `final_status` and the live server side decision matrix's `final_status` are computed differently, the former purely from distance, the latter from distance combined with the pattern anomaly flag, as shown directly in Figure 5. This is intentional and documented in the code, but it does mean the two classification paths are not strictly comparable label for label without accounting for that difference.
- In the live test session captured for this report, seventeen of eighteen logged TinyML predictions were flagged as pattern anomalies, because the detections were triggered manually at irregular intervals rather than by a continuously moving object sampled every second. This says more about the artificial nature of manual button driven testing than about the pattern check itself, and a longer session with a genuinely moving object would be expected to show a healthier mix.
- No formal comparison has yet been made between the server side decision matrix's classifications and the on device Decision Tree's classifications for the same live events, since the two are logged to separate tables and the dataset used to train the tree does not share the matrix's escalation rule.
- As in Phase 3, the expected sampling interval and its twenty five percent tolerance remain reasonable starting values rather than figures derived from a formal statistical study of real object movement patterns.

10. Future Work

The clearest next step is to retrain the Decision Tree on labels that match the live server side decision matrix rather than a purely distance based rule, which would let the pattern anomaly and interval features actually influence the model's predictions and would make the on device classifier a genuine embedded approximation of the DBMS logic rather than a rediscovery of the distance thresholds alone. Once that retraining is done, a direct, row by row comparison between `detection_events.final_status` and `tinyml_predictions.predicted_severity` for the same physical events would give a meaningful measure of how well the embedded model actually approximates the server side decision, rather than only reporting the offline test accuracy against its own training labels.

A longer duration TinyML mode test session, using a continuously moving object sampled at the full one second cadence rather than manually triggered events, would give a more representative picture of how often the pattern anomaly check fires under realistic conditions, and would produce a larger, more balanced live prediction log to evaluate. Beyond that, the same future work items already identified in Phase 3 remain relevant, adding database transactions so a detection event and its sensor readings are always saved together, and formally

evaluating the existing Isolation Forest models with precision and recall once enough labeled anomaly data has accumulated.

11. References

- [1] Espressif Systems, "ESP-NOW User Guide," GitHub. [Online]. Available: https://github.com/espressif/esp-now/blob/master/User_Guide.md
- [2] Espressif Systems, "ESP-NOW API Reference," ESP-IDF Programming Guide. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html
- [3] Espressif Systems, "ESP8266 Non-OS SDK Programming Guide." [Online]. Available: <https://docs.espressif.com/projects/esp8266-rtos-sdk/en/latest/>
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- [5] scikit-learn developers, "sklearn.tree.DecisionTreeClassifier," scikit-learn Documentation. [Online]. Available: <https://scikitlearn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- [6] S. Salerno, "micromlgen: Generate C Code for Microcontrollers from Python's sklearn Classifiers," GitHub. [Online]. Available: <https://github.com/eloquentarduino/micromlgen>
- [7] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM), Pisa, Italy, 2008, pp. 413-422. [Online]. Available: <https://ieeexplore.ieee.org/document/4781136>
- [8] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," Communications of the ACM, vol. 13, no. 6, pp. 377-387, Jun. 1970. [Online]. Available: <https://dl.acm.org/doi/10.1145/362384.362685>
- [9] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Hoboken, NJ, USA: Pearson, 2021.
- [10] IEEE Standards Association, "IEEE Standard for Information Technology-Telecommunications and Information Exchange Between Systems, Local and Metropolitan Area Networks," IEEE Std 802.11-2012, 2012. [Online]. Available: <https://standards.ieee.org/ieee/802.11/4149/>
- [11] Chart.js Contributors, "Chart.js Documentation." [Online]. Available: <https://www.chartjs.org/docs/latest/>
- [12] Tecnick.com, "TCPDF: PHP Library for Generating PDF Documents." [Online]. Available: <https://github.com/tecnickcom/TCPDF>